Heart Disease Risk Prediction

An End-to-End MLOps Pipeline

Course	Machine Learning Operations (MLOps) – AIMLCZG523
Assignment	Assignment 01 – End-to-End ML Development, CI/CD & Deployment
Student Name	Ritu Mishra
Student ID	2025ae05964
Institute	BITS Pilani – Work Integrated Learning Programmes
Submission Date	20 October 2026

Repository: https://github.com/2025ae05964/mlops-heart-disease-prediction

1. Introduction and Objective

This report documents the design, implementation and deployment of an end-to-end Machine Learning Operations (MLOps) pipeline for predicting the presence or absence of heart disease from patient clinical data. The objective of the assignment is to move beyond offline model development and to realise a reproducible, automated and monitored solution that mirrors a real-world production workflow. Accordingly, the project integrates data acquisition and exploratory analysis, model development with hyper-parameter tuning, experiment tracking, model packaging, automated testing through continuous integration, containerisation, orchestration on Kubernetes, and runtime monitoring.

The problem is framed as a binary classification task: given a vector of clinical attributes for a patient, the system predicts whether heart disease is present and returns an associated confidence score through a REST API. The emphasis throughout is on engineering discipline – reproducibility, automation, isolation and observability – rather than on the predictive model alone.

2. Setup and Installation

The project is implemented in Python and is fully reproducible from a clean environment using the pinned dependency file. The core workflow is executed as follows:

```
python -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt
python src/data_download.py --source local --in data/processed.cleveland.data --out
data/heart_raw.csv
python src/eda.py
python src/train.py
mlflow ui   # experiment dashboard at http://127.0.0.1:5000
```

All dependencies are declared in **requirements.txt** with explicit version pins to guarantee deterministic behaviour across machines. Every stage of the pipeline is reproducible because a fixed random seed (random_state = 42) is used for data splitting, cross-validation and model training. The repository is organised into clearly separated modules: **src/** for data, pre-processing, EDA and training logic; **app/** for the serving API; **tests/** for unit tests; **k8s/** for deployment manifests; and **monitoring/** for the observability stack.

3. Dataset and Exploratory Data Analysis

3.1 Dataset Description

The study uses the Heart Disease dataset from the UCI Machine Learning Repository (Cleveland subset). It comprises 303 patient records described by 13 clinical predictors and one target attribute. The predictors include demographic and physiological measurements such as age, sex, chest-pain type (cp), resting blood pressure (trestbps), serum cholesterol (chol), fasting blood sugar (fbs), resting electrocardiographic results (restecg), maximum heart rate achieved (thalach), exercise-induced angina (exang), ST depression (oldpeak), the slope of the peak exercise ST segment (slope), the number of major vessels (ca) and the thalassemia indicator (thal).

The original target attribute *num* records disease severity on a scale from 0 to 4. In accordance with the binary problem statement, it is mapped to two classes: 0 denotes the absence of disease and 1 denotes its presence (severity ≥ 1). Six records contain missing values, encoded in the raw file by the '?' character in the *ca* and *thal* columns; these are converted to proper missing markers during cleaning and subsequently imputed.

3.2 Distribution of Numeric Features

Histograms of the continuous predictors were generated to understand their distributions, scales and skew prior to modelling. These confirm that the features operate on markedly different scales (for example, cholesterol in the hundreds versus ST depression below four), motivating the standardisation step applied during feature engineering.

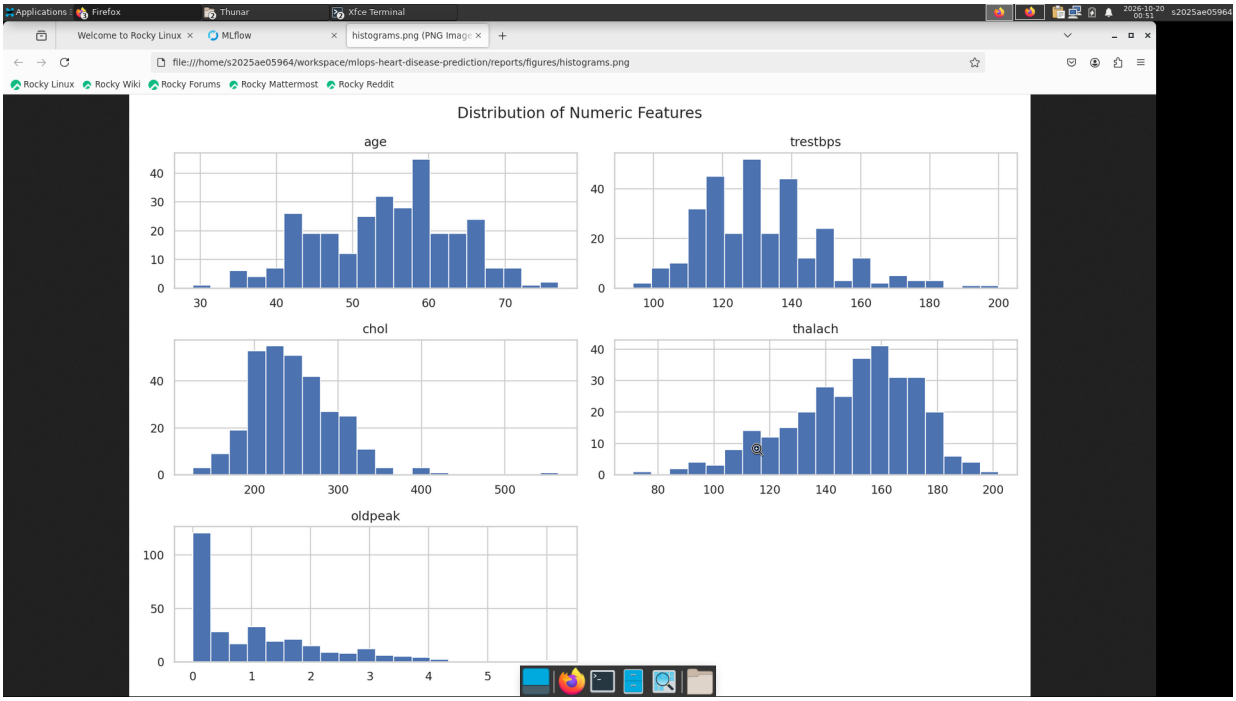


Figure 2. Histograms of the numeric clinical features.

3.3 Feature Correlation

A correlation heat-map was computed over the numeric predictors and the binary target to identify the attributes most associated with heart disease. Maximum heart rate achieved (thalach) exhibits a negative association with the target, whereas ST depression (oldpeak) and age show positive associations. No pair of predictors is so strongly correlated as to introduce serious multicollinearity, so all features are retained for modelling.



Figure 3. Correlation heat-map of numeric features and the target.

3.4 Class Balance

The distribution of the binary target was examined to determine whether class imbalance would bias training. The dataset is well balanced, with 164 records corresponding to the absence of disease and 139 to its presence. Because the classes are close to parity, accuracy remains a meaningful summary metric, although precision, recall and ROC-AUC are also reported to provide a complete picture of classifier behaviour in a clinical screening context. The two classes are reasonably close in size, with class 0 containing 25 more records than class 1; the dataset is therefore not perfectly balanced, which is a further reason to report precision and recall alongside accuracy.

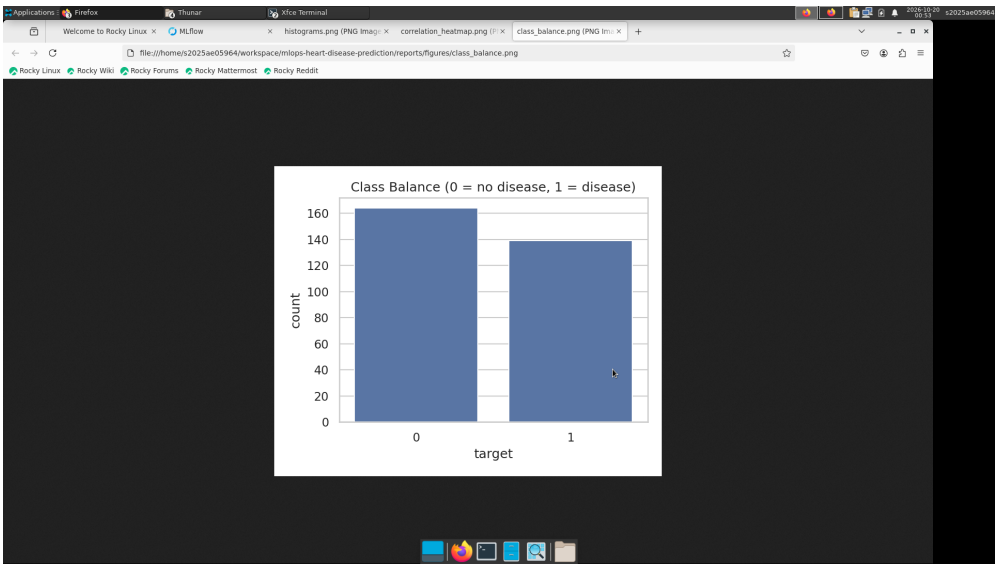


Figure 4. Class balance of the binary target (0 = no disease, 1 = disease).

4. Feature Engineering and Model Development

4.1 Pre-processing Pipeline

To guarantee that identical transformations are applied during training and serving, all pre-processing is encapsulated in a single scikit-learn **ColumnTransformer** embedded within a **Pipeline**. The five continuous features are median-imputed and standardised with **StandardScaler**; the eight categorical features are imputed with their most frequent value and one-hot encoded with **handle_unknown='ignore'** so that unseen categories at inference time do not cause failures. Fitting the transformer inside the pipeline ensures that scaling and encoding parameters are learned only from the training partition, thereby preventing data leakage.

4.2 Models, Tuning and Evaluation

Two complementary classifiers were trained: Logistic Regression, a linear and highly interpretable baseline, and Random Forest, a non-linear ensemble capable of capturing feature interactions. Each model was wrapped in the shared pre-processing pipeline and tuned with **GridSearchCV** using five-fold cross-validation, with the mean ROC-AUC as the selection criterion. For Logistic Regression the inverse regularisation strength C was searched over {0.1, 1.0, 10.0}; for Random Forest the number of estimators and maximum depth were searched over {100, 300} and {None, 5, 10} respectively. Models were evaluated on a held-out, stratified 20% test partition that was never seen during tuning.

Model	Accuracy	Precision	Recall	ROC-AUC	CV Acc. (mean)
Logistic Regression	0.885	0.839	0.929	0.965	0.835
Random Forest	0.869	0.833	0.893	0.948	0.806

Table 1. Held-out test performance of the two classifiers (seed = 42).

Logistic Regression was selected as the production model because it achieved the highest ROC-AUC (0.965) together with the strongest recall (0.929). In a clinical screening setting, high recall is particularly desirable because it minimises false negatives – that is, cases where a patient with disease is incorrectly classified as healthy. The selected configuration uses C = 0.1. The linear model additionally offers greater transparency than the ensemble, which is advantageous for clinical interpretability.

5. Experiment Tracking with MLflow

Experiment tracking was implemented with MLflow. Each training run is wrapped in an MLflow run context under the experiment *heart-disease-classification*. For every model the best hyper-parameters, the full set of evaluation metrics (accuracy, precision, recall, ROC-AUC and mean cross-validation accuracy), the serialised model artifact and the EDA figures are logged. This provides a complete, queryable audit trail of every experiment and enables direct comparison between the candidate models.

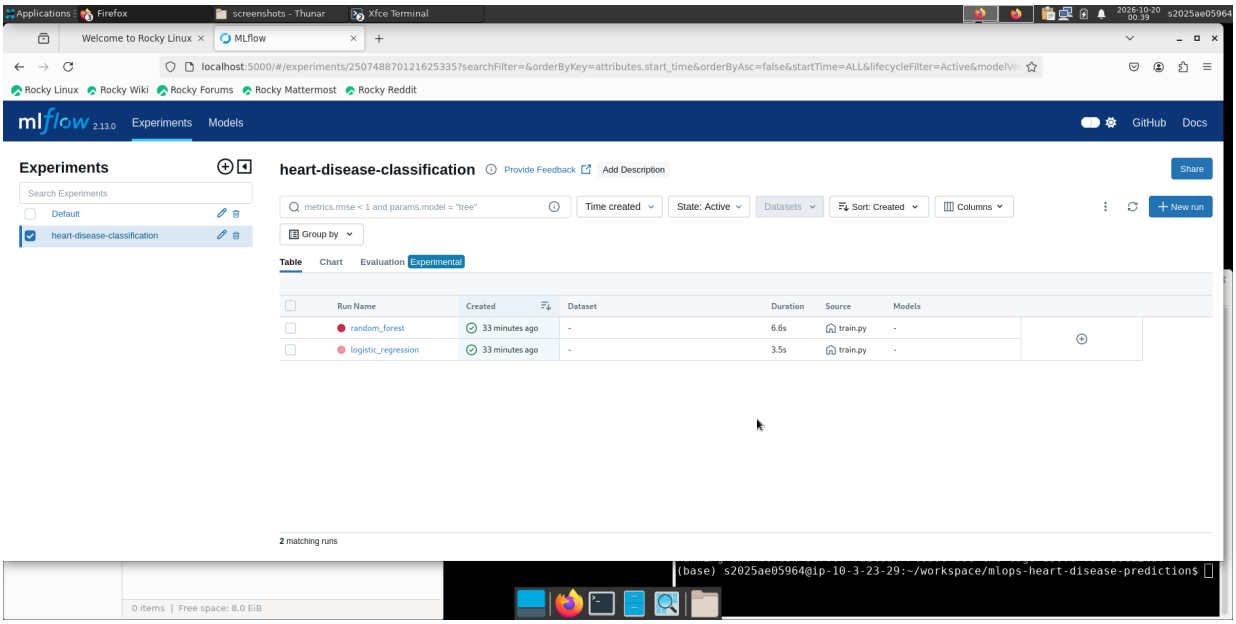


Figure 5. MLflow experiment view listing the tracked training runs.

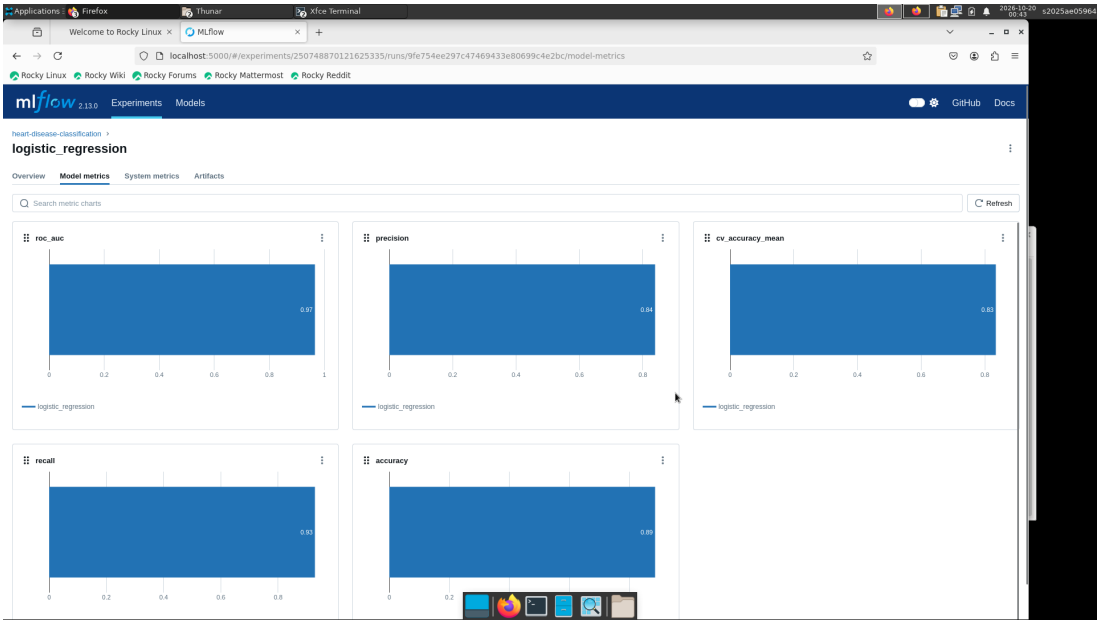


Figure 6. Logged metrics for the Logistic Regression run (ROC-AUC 0.97, precision 0.84, recall 0.93, accuracy 0.89, CV accuracy 0.83).

6. Model Packaging and Reproducibility

The selected pipeline is persisted in its entirety – pre-processing transformers and the fitted estimator together – as **models/model.joblib**. Because the serialised object is the complete pipeline, the serving layer applies exactly the same transformations that were used during training, eliminating train–serve skew. Reproducibility is further supported by the pinned **requirements.txt**, the fixed random seed, and a deterministic data-preparation script, so that the entire experiment can be regenerated from a clean checkout.

7. Automated Testing and CI/CD

A suite of unit tests under **tests/** validates the data-processing and modelling logic, covering target binarisation, correct handling of the '?' missing markers, and the numeric integrity of the transformed feature matrix. A GitHub Actions workflow (**.github/workflows/ci.yml**) runs automatically on every push and executes the pipeline in the mandated order: code linting with flake8, data preparation, unit testing with pytest, and model training, after which the trained model and reports are uploaded as build artifacts. The workflow is configured to fail on any linting, test or training error and to surface clear logs, satisfying the production-readiness requirement that the pipeline must fail fast on faults.

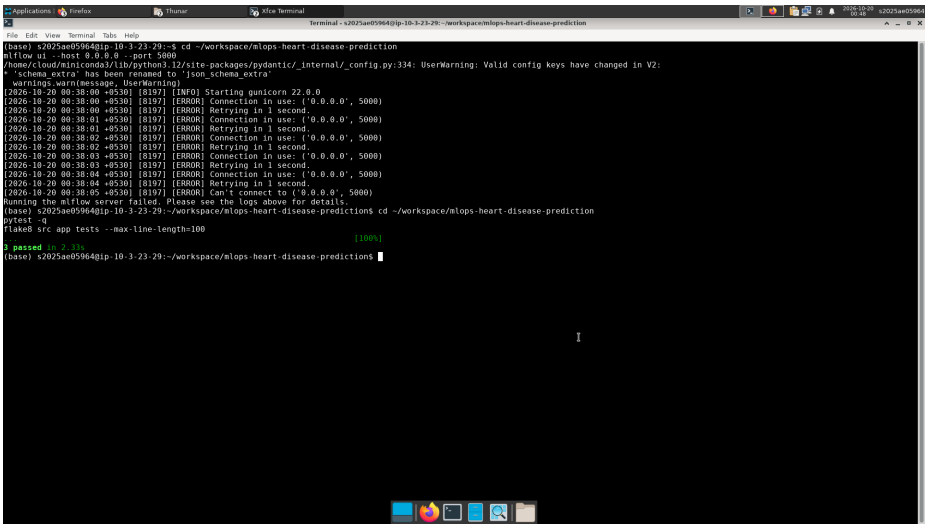


Figure 7. Successful unit-test execution and lint check.

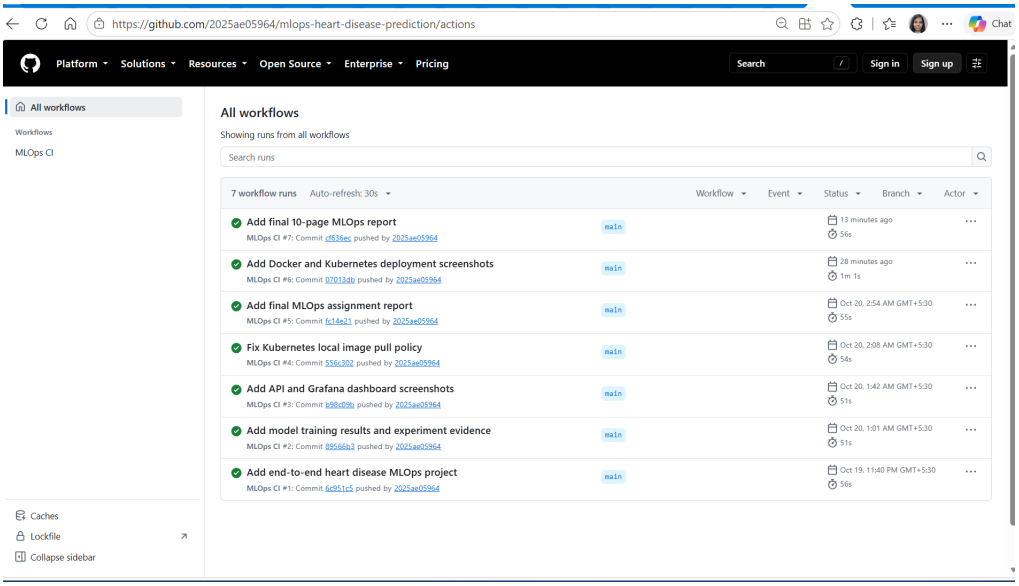


Figure 8. GitHub Actions workflow history: all seven CI runs completed successfully (lint, unit tests and model training) on every push.

8. System Architecture

The overall architecture integrates the model lifecycle with the operational infrastructure. Raw data flows through pre-processing into model training; experiments are tracked in MLflow; the best model is packaged and exposed through a FastAPI service; the service is containerised with Docker and orchestrated on a local Kubernetes cluster with two replicas behind a LoadBalancer service; and runtime behaviour is observed through Prometheus and Grafana. The entire software lifecycle is governed by the GitHub Actions CI/CD pipeline. Figure 8 summarises these relationships.

Figure 1. End-to-End MLOps Architecture

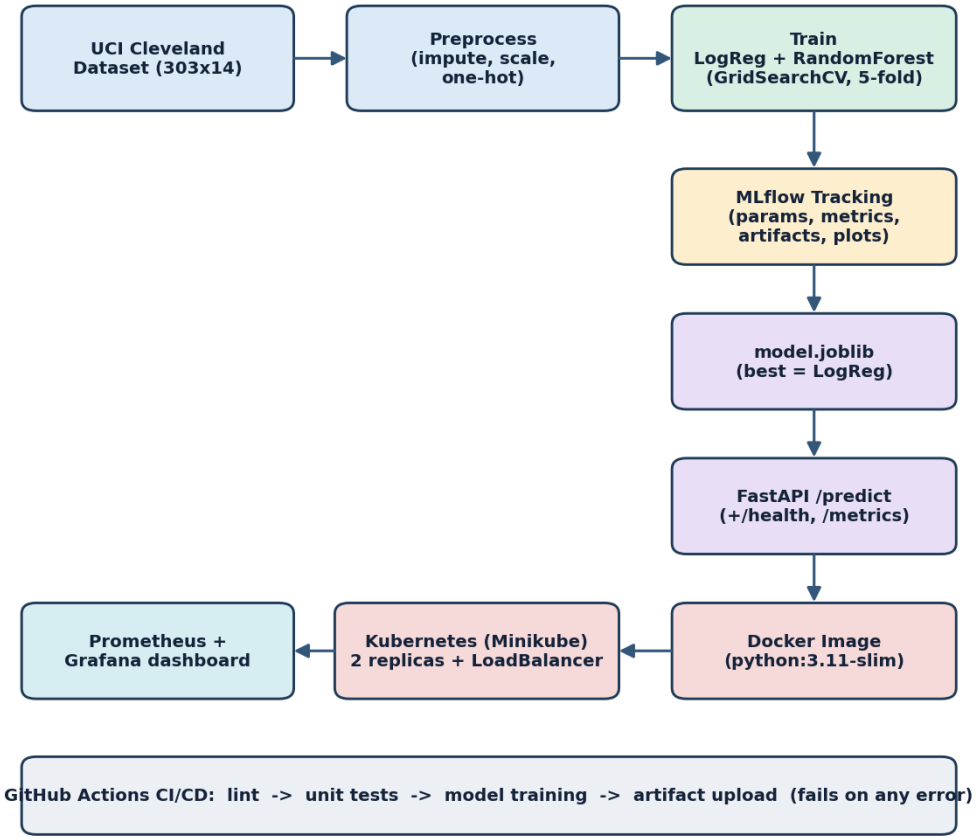


Figure 9. End-to-end MLOps architecture of the deployed solution.

9. Containerisation and Serving API

The model is served by a FastAPI application exposing three endpoints: **/health** for liveness and readiness checks, **/predict** which accepts a JSON patient record and returns the predicted class, a human-readable label and a confidence score, and **/metrics** for metrics scraping. The application is containerised using a lightweight **python:3.11-slim** base image; dependencies are installed from the pinned requirements file and the trained model is copied into the image so that the container is fully self-contained. The container builds and runs in an isolated environment and returns correct predictions for sample input, fulfilling the containerisation requirement.

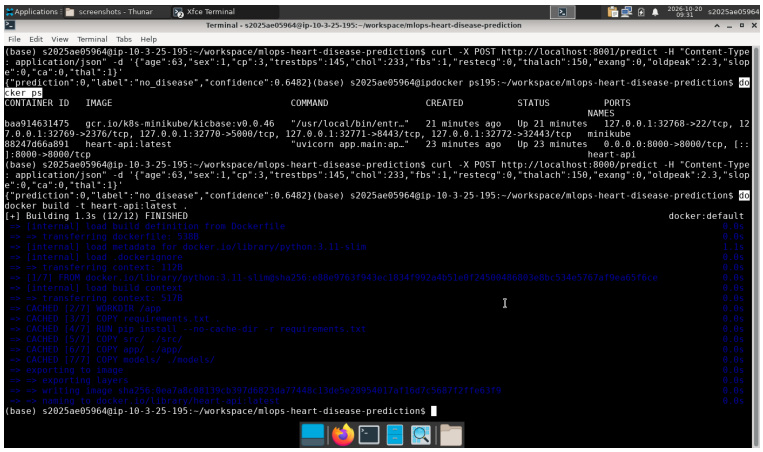


Figure 10. Successful Docker image build.

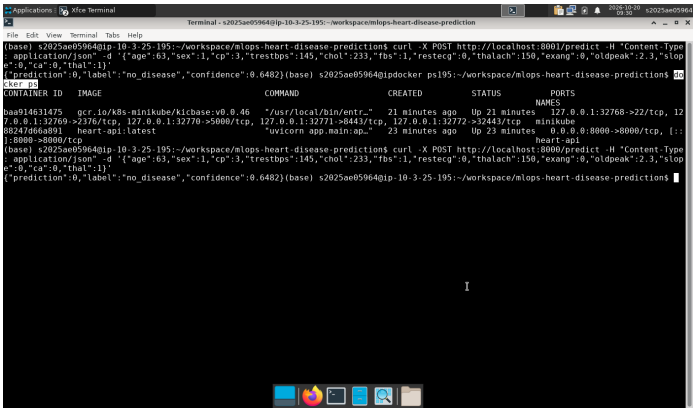


Figure 11. Prediction returned by the containerised /predict endpoint.

10. Production Deployment on Kubernetes

The container was deployed to a local Kubernetes cluster provisioned with Minikube. The deployment manifest specifies two replicas for availability, a readiness probe against the /health endpoint, and resource requests and limits; a companion Service of type LoadBalancer exposes the API. The running pods and service were verified, and the /predict endpoint was exercised through the service to confirm correct end-to-end operation inside the cluster.

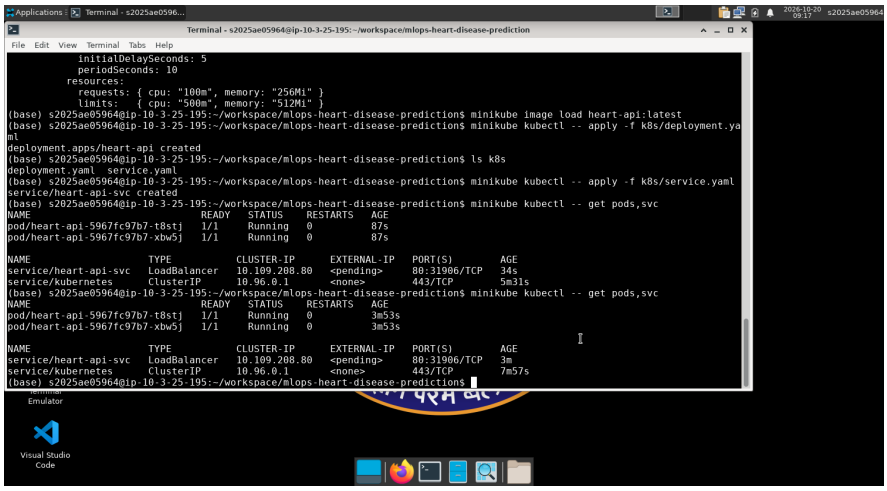


Figure 12. Running pods and exposed service on the Kubernetes cluster.

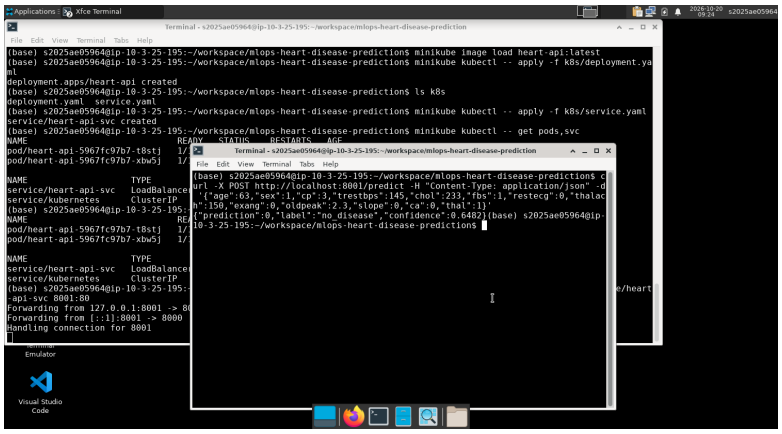


Figure 13. Prediction served through the Kubernetes LoadBalancer service.

11. Monitoring and Logging

Observability is provided through request logging and a metrics pipeline. The FastAPI service logs each prediction request, and the Prometheus FastAPI instrumentator exposes operational metrics (request counts, latencies and status codes) at the /metrics endpoint. A Docker Compose stack runs the API alongside Prometheus, which scrapes these metrics, and Grafana, which visualises them on a dashboard. This demonstrates a minimal but representative production monitoring setup.

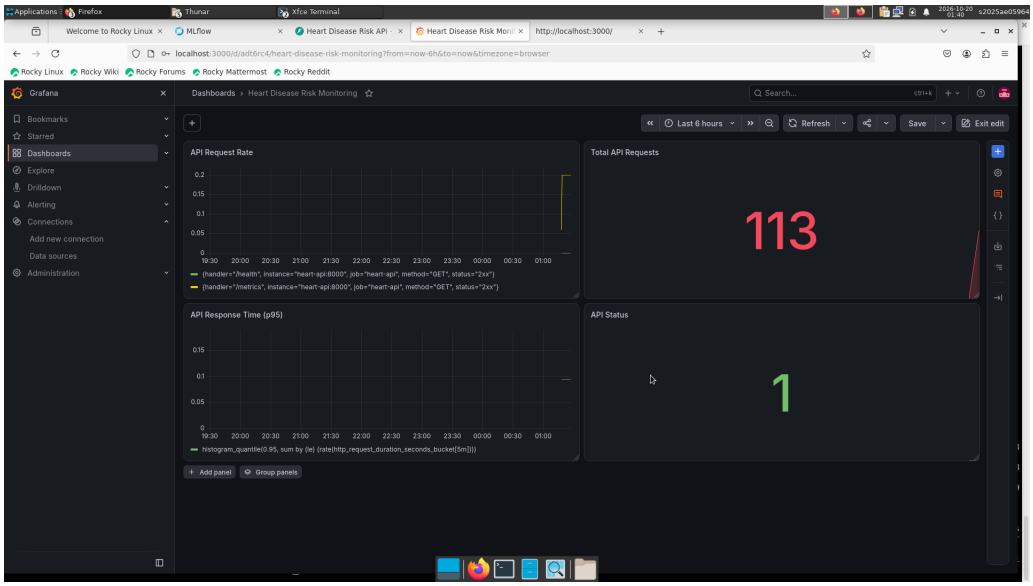


Figure 14. Grafana dashboard visualising API metrics scraped by Prometheus.

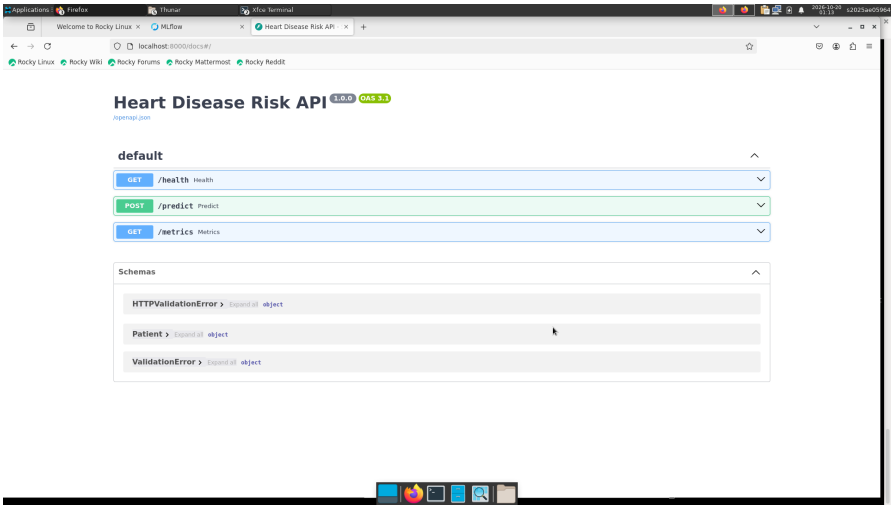


Figure 15. Interactive FastAPI (Swagger) documentation of the serving API.

12. Conclusion

This project delivered a complete, reproducible and monitored MLOps pipeline for heart-disease risk prediction. Logistic Regression was selected as the production model, achieving a ROC-AUC of 0.965 and a recall of 0.929 on a held-out test set. Beyond the model, the work demonstrates the operational practices that distinguish MLOps from offline machine learning: deterministic reproducibility, automated testing and continuous integration, containerised and orchestrated deployment, and runtime monitoring. Possible extensions include probability calibration for more reliable clinical risk estimates, persistent monitoring storage, authenticated and TLS-secured endpoints, and automated model retraining triggered by data drift.

References

[1] A. Janosi, W. Steinbrunn, M. Pfisterer, and R. Detrano, “Heart Disease,” UCI Machine Learning Repository, 1989. doi: 10.24432/C52P4X.

[2] F. Pedregosa et al., “Scikit-learn: Machine Learning in Python,” Journal of Machine Learning Research, vol. 12, pp. 2825–2830, 2011.

[3] M. Zaharia et al., “Accelerating the Machine Learning Lifecycle with MLflow,” IEEE Data Engineering Bulletin, vol. 41, no. 4, pp. 39–45, 2018.

[4] D. Sculley et al., “Hidden Technical Debt in Machine Learning Systems,” in Advances in Neural Information Processing Systems (NeurIPS), 2015.

[5] Repository: https://github.com/2025ae05964/mlops-heart-disease-prediction